Starter: What does this code output? Explain step-by-step

```
List<string> myNames, yourNames;
myNames = new List<string>();
yourNames = new List<string>();

myNames.Add("Matt");
myNames.Add("Chris");

yourNames = myNames;

myNames.Remove("Matt");
myNames.Add("Isabel");

Console.WriteLine(myNames[0] + ", " + myNames[1]);
Console.WriteLine(yourNames[0] + ", " + yourNames[1]);

Console.ReadKey();
```

Starter: What does this code output? Explain step-by-step

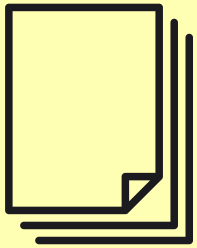
```
List<string> myNames, yourNames;
myNames = new List<string>();
yourNames = new List<string>();

myNames.Add("Matt");
myNames.Add("Chris");

yourNames = myNames;

myNames.Remove("Matt");
myNames.Add("Isabel");

Console.WriteLine(myNames[0] + ", " + myNames[1]);
Console.WriteLine(yourNames[0] + ", " + yourNames[1]);

Console.ReadKey();
```

```
Chris, Isabel
Chris, Isabel
```

Function Stubs

Keywords: `Function; stub; parameter; return value; ref;`

Function Stubs



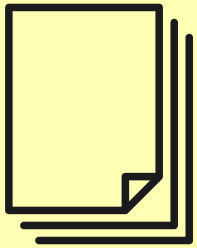
Function Stubs are a template for a function.

Great when programming for identifying features that you're going to have, but don't have yet.

They contain:

- The static keyword (if in `Program`)

Function Stubs



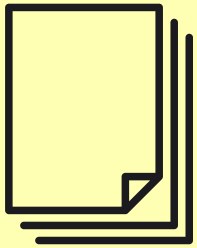
Function Stubs are a template for a function.

Great when programming for identifying features that you're going to have, but don't have yet.

They contain:

- The static keyword (if in `Program`)
- The return type

Function Stubs



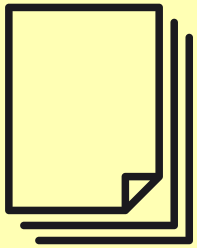
Function Stubs are a template for a function.

Great when programming for identifying features that you're going to have, but don't have yet.

They contain:

- The static keyword (if in `Program`)
- The return type
- An identifier

Function Stubs



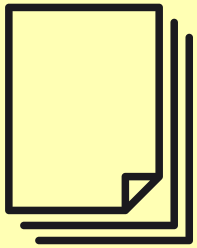
Function Stubs are a template for a function.

Great when programming for identifying features that you're going to have, but don't have yet.

They contain:

- The static keyword (if in `Program`)
- The return type
- An identifier
- The parameter list

Function Stubs



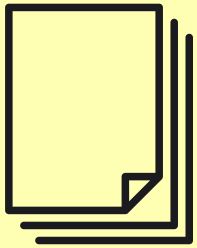
Function Stubs are a template for a function.

Great when programming for identifying features that you're going to have, but don't have yet.

They contain:

- The static keyword (if in `Program`)
- The return type
- An identifier
- The parameter list
- (optional) A basic return value

Function Stubs

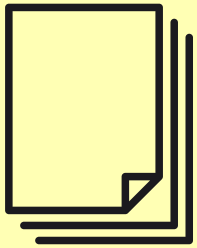


Function Stubs are a template for a function.

Great when programming for identifying features that you're going to have, but don't have yet.

They do not contain the final implementation of the function.

Function Stubs



Function Stubs are a template for a function.

Great when programming for identifying features that you're going to have, but don't have yet.

They do not contain the final implementation of the function.

They do have the correct **interface**

Write the function stubs only.

1. Create a function called `Subtract`. It should take as parameter an integer called num1. It should take as second parameter an integer called num2. It should return an integer.
2. Create a function called `SortList`. It should take as parameter a list of integers to sort. It should return a new list containing the numbers in ascending order.
3. Create a function called `DecToHex`. It should take as input a whole number (possibly negative) and convert it into hexadecimal. The result should be returned in the form "`0x5A81FF`".
4. Create a function called `RemoveStudent`. It should remove the student's name from the specified **Lesson**'s student list. It should return True if a student was removed, and False otherwise

Identify the blanks

```
struct Lesson
{
    pu[ 1 ]om, teacher;
    public List<string> students;
}
static bool removeStudent(string name, Lesson less)
{
    if (less.students.Contains(name))
    {
        [ 2 ]  [ 3 ]
        less.students.Remove(name);
        [ 4 ]  [ 5 ]
        [ 6 ]
    }
    else
    {
        return false;
    }
}
```

Identify the blanks

```
struct Lesson
{
    public string room, teacher;
    public List<string> students;
}
static bool removeStudent(string name, Lesson less)
{
    if (less.students.Contains(name))
    {
        less.students.Remove(name);
    }
    else
    {
        return false;
    }
}
```

2 3 4 5 6

Identify the blanks

```
struct Lesson
{
    public string room, teacher;
    public List<string> students;
}
static bool removeStudent(string name, Lesson less)
{
    if (less.students.Contains(name))
    {                                  3
        less.students.Remove(name);
        4                      5
                6
    }
    else
    {
        return false;
    }
}
```

Identify the blanks

```
struct Lesson
{
    public string room, teacher;
    public List<string> students;
}
static bool removeStudent(string name, Lesson less)
{
    if (less.students.Contains(name))
    {
        less.students.Remove(name);
    }
    else
    {
        return false;
    }
}
```

4 5 6

Identify the blanks

```
struct Lesson
{
    public string room, teacher;
    public List<string> students;
}
static bool removeStudent(string name, Lesson less)
{
    if (less.students.Contains(name))
    {
        less.students.Remove(name);
        return true;
    }
    else
    {
        return false;
    }
}
```

5

6

Identify the blanks

```
struct Lesson
{
    public string room, teacher;
    public List<string> students;
}
static bool removeStudent(string name, Lesson less)
{
    if (less.students.Contains(name))
    {
        less.students.Remove(name);
        return true;
    }
    else
    {
        return false;
    }
}
```

6

Identify the blanks

```
struct Lesson
{
    public string room, teacher;
    public List<string> students;
}
static bool removeStudent(string name, Lesson less)
{
    if (less.students.Contains(name))
    {
        less.students.Remove(name);
        return true;
    }
    else
    {
        return false;
    }
}
```

Passing Parameters

Keywords: Subroutine; parameter; passing parameters; call; calling code; by value; by reference; ref;

What does this code output?

```
static void Main(string[] args)
{
    int x = 4;
    doubler(x);
    Console.WriteLine(x);
    Console.ReadKey();
}
static void doubler(int x)
{
    x = x * 2;
    Console.WriteLine(x);
}
```

What does this code output?

```
static void Main(string[] args)
{
    int x = 4;
    doubler(x);
    Console.WriteLine(x);
    Console.ReadKey();
}
static void doubler(int x)
{
    x = x * 2;
    Console.WriteLine(x);
}
```

```
8
4
```

What does this code output?

```
static void Main(string[] args)
{
    int x = 4;
    doubler(ref x);
    Console.WriteLine(x);
    Console.ReadKey();
}
static void doubler(ref int x)
{
    x = x * 2;
    Console.WriteLine(x);
}
```

What does this code output?

```
static void Main(string[] args)
{
    int x = 4;
    doubler(ref x);
    Console.WriteLine(x);
    Console.ReadKey();
}
static void doubler(ref int x)
{
    x = x * 2;
    Console.WriteLine(x);
}
```

```
8
8
```

Wait, What?

The difference:

Original:

Call:

```
doubler(x);
```

Method:

```
doubler(int x)
```

Changed:

Call:

```
doubler(ref x);
```

Method:

```
doubler(ref int x)
```

Wait, What?

The difference:

Original:

Call:

```
doubler(x);
```

Method:

```
doubler(int x)
```

Changed:

Call:

```
doubler(ref x);
```

Method:

```
doubler(ref int x)
```

So what is "`ref`"?

The "`ref`" keyword

Whenever we pass a **primitive data type** into a function, we pass what's in the stack.

Whenever we pass a **reference**, we pass a pointer to that memory location.

Let's try the original again, but with some labels.

What does this code output?

```
static void Main(string[] args)
{
    int x = 4;
    doubler(x);
    Console.WriteLine(x);
    Console.ReadKey();
}
static void doubler(int x)
{
    x = x * 2;
    Console.WriteLine(x);
}
```

What does this code output?

```
static void Main(string[] args)
{
    int x = 4;
    doubler(x);
    Console.WriteLine(x);
    Console.ReadKey();
}
static void doubler(int x)
{
    x = x * 2;
    Console.WriteLine(x);
}
```

```
8
4
```

What does this code output?

```
static void Main(string[] args)
{
    int x = 4;
    doubler(ref x);
    Console.WriteLine(x);
    Console.ReadKey();
}
static void doubler(ref int x)
{
    x = x * 2;
    Console.WriteLine(x);
}
```

What does this code output?

```
static void Main(string[] args)
{
    int x = 4;
    doubler(ref x);
    Console.WriteLine(x);
    Console.ReadKey();
}
static void doubler(ref int x)
{
    x = x * 2;
    Console.WriteLine(x);
}
```

```
8
8
```

You Try:

```
static void Add1Val(int input)
{
    input += 1;
}
static void Main(string[] args)
{
    int num = 5;
    Console.WriteLine(num);
    Add1Val(num);
    Console.WriteLine(num);

    Console.ReadKey();
}
```

You Try:

```
static void Add1Val(int input)
{
    input += 1;
}
static void Main(string[] args)
{
    int num = 5;
    Console.WriteLine(num);
    Add1Val(num);
    Console.WriteLine(num);

    Console.ReadKey();
}
```

```
5
5
```

You Try:

```
static void Add1Ref(ref int input)
{
    input += 1;
}
static void Main(string[] args)
{
    int num = 5;
    Console.WriteLine(num);
    Add1Ref(ref num);
    Console.WriteLine(num);

    Console.ReadKey();
}
```

You Try:

```
static void Add1Ref(ref int input)
{
    input += 1;
}
static void Main(string[] args)
{
    int num = 5;
    Console.WriteLine(num);
    Add1Ref(ref num);
    Console.WriteLine(num);

    Console.ReadKey();
}
```

```
5
6
```

You Try:

```
static void ChangeVal(int[] input)
{
   input[0] += 1;
}
static void ChangeRef(ref int[] input)
{
   input[0] += 1;
}
static void Main(string[] args)
{
   int[] nums = new int[1];
   nums[0] = 5;

   Console.WriteLine(nums[0]);
   ChangeVal(nums);
   Console.WriteLine(nums[0]);
   ChangeRef(ref nums);
   Console.WriteLine(nums[0]);

   Console.ReadKey();
}
```

You Try:

```
static void ChangeVal(int[] input)
{
   input[0] += 1;
}
static void ChangeRef(ref int[] input)
{
   input[0] += 1;
}
static void Main(string[] args)
{
   int[] nums = new int[1];
   nums[0] = 5;

   Console.WriteLine(nums[0]);
   ChangeVal(nums);
   Console.WriteLine(nums[0]);
   ChangeRef(ref nums);
   Console.WriteLine(nums[0]);

   Console.ReadKey();
}
```

```
5
6
7
```

Practice!

1. Complete this code:

```
static void Add(ref int answer, int num1, int num2)
{
    // To do: complete procedure body
}

static void Main(string[] args)
{
    int result = 0;
    //to do: call Add procedure
    Console.WriteLine(result);
    Console.ReadKey();
}
```

Extension: Make a procedure called `Swapper` which swaps the values of two variables passed to it by reference.

2. Create a function called `Divide`, similar to `Add`, but which also returns a **Boolean** value indicating whether the division was possible or not (divide by zero)